Data Structures with Python

Viva Questions and Answers (30 questions)

1. What is a data structure?

Ans: A way of storing and organising data so that we can use it easily and quickly. Example: list, stack, queue.

2. Which built-in data structures are there in Python?

Ans: List, tuple, set, dictionary and string.

3. What is the difference between a list and a tuple?

Ans: A list can be changed after creation (mutable). A tuple cannot be changed (immutable).

4. What is a set? How do you create an empty set?

Ans: A set is a collection of unique items with no fixed order. Duplicates are removed automatically. An empty set is made with set(), because {} makes an empty dictionary.

5. What is the difference between remove() and discard()?

Ans: If the item is not in the set, remove() gives a KeyError, but discard() does nothing.

6. What is a dictionary?

Ans: It stores data as key-value pairs, like a word and its meaning. Finding a value by its key is very fast (O(1) on average).

7. What is the difference between append() and insert()?

Ans: append() adds an item at the end of the list. insert() adds an item at the position you choose.

8. What is the difference between sort() and sorted()?

Ans: sort() arranges the same list and returns None. sorted() returns a new sorted list and keeps the original safe.

9. Are strings mutable in Python?

Ans: No. Strings are immutable. Every string method gives a new string and the old one stays the same.

10. What is the difference between find() and index()?

Ans: If the text is not found, find() returns -1, but index() gives a ValueError.

11. What is an exception?

Ans: An error that happens while the program is running and stops its normal flow. Example: dividing by zero (ZeroDivisionError).

12. What are try, except, else and finally?

Ans: try holds the risky code. except handles the error. else runs only if there was no error. finally always runs.

13. How do you create your own exception?

Ans: Make a class that inherits from Exception, then use the raise keyword to trigger it.

14. What is recursion?

Ans: When a function calls itself to solve a smaller part of the same problem. Example: factorial.

15. What is a base case? Why is it needed?

Ans: It is the condition that stops the recursion. Without it the function calls itself forever and Python gives RecursionError.

16. How many moves are needed in Tower of Hanoi for n disks?

Ans: 2^n - 1 moves. For 3 disks it is 7 moves.

17. What is an Abstract Data Type (ADT)?

Ans: It tells what operations are available, but not how they are done inside. Example: a stack has push and pop.

18. How do you make an abstract class in Python?

Ans: Inherit from ABC and mark methods with @abstractmethod. You cannot create an object of an abstract class directly.

19. What is a stack? Give some uses.

Ans: A stack works as LIFO (Last In, First Out). Insert and delete happen only at the top. Uses: undo/redo, browser back button, function calls, bracket checking.

20. What are stack overflow and underflow?

Ans: Overflow is pushing into a full stack. Underflow is popping from an empty stack.

21. What is a queue?

Ans: A queue works as FIFO (First In, First Out). Items are added at the rear and removed from the front. Example: printer queue.

22. Why do we use deque instead of a list for a queue?

Ans: deque removes from the front in O(1) time. list.pop(0) takes O(n) because all other items shift.

23. What is a linked list?

Ans: A chain of nodes. Each node stores data and a link (pointer) to the next node.

24. What is the advantage of a linked list over an array?

Ans: Its size can grow or shrink easily, and insertion or deletion needs no shifting of elements.

25. What is the difference between a singly and a doubly linked list?

Ans: A doubly linked list node has an extra prev pointer, so we can move both forward and backward.

26. What is a tree? Define root, leaf and height.

Ans: A tree is a hierarchical structure of nodes. Root is the top node. Leaf is a node with no children. Height is the length of the longest path from root to a leaf.

27. What is the rule of a Binary Search Tree (BST)?

Ans: For every node, smaller keys go to the left subtree and larger keys go to the right subtree.

28. What are the types of tree traversal?

Ans: Depth-first: inorder (Left, Root, Right), preorder (Root, Left, Right), postorder (Left, Right, Root). Breadth-first: level order, using a queue.

29. What does inorder traversal of a BST give?

Ans: The keys in sorted (ascending) order. For our tree: 20, 30, 40, 50, 60, 70, 80.

30. What is the time complexity of searching in a BST?

Ans: O(log n) on average when the tree is balanced, and O(n) in the worst case when the tree is skewed like a line.